

Momento II – Diseño lógico y visual

1. Vistas, interacciones y físicas

Nivel 1 – “Evacuación”

Vista: cenital (desde arriba).

Objetivo: guiar al personaje hasta el refugio antes de que se acabe el tiempo.

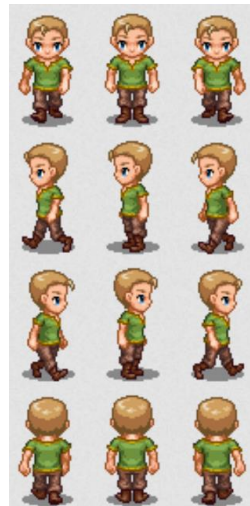
Interacciones: el jugador puede moverse en 4 direcciones. Si toca el fuego o los escombros, pierde tiempo.

Físicas:

1. Movimiento rectilíneo uniforme (jugador).
2. Movimiento parabólico (escombros que caen).
3. Oscilación simple (humo decorativo).

Agente autónomo: un perro guía que detecta zonas de fuego (percepción), busca un camino libre (razonamiento), ladra para guiar al jugador (acción) y recuerda las zonas seguras (aprendizaje).

Dibujo informal:



Vista: lateral fija (2D).

Objetivo: calmar la multitud antes de que aumente demasiado la tensión.

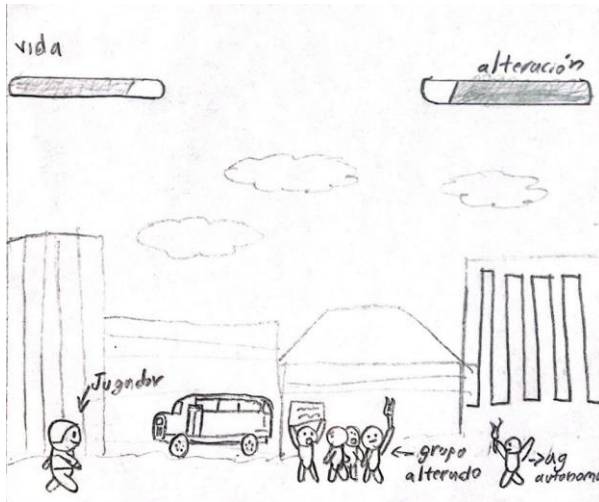
Interacciones: el jugador lanza mensajes (con clic o tecla).

Físicas:

1. Movimiento aleatorio de la multitud.
2. Movimiento amortiguado de pancartas (balanceo).
3. Movimiento uniforme del jugador.

Agente autónomo: manifestante neutral que percibe el ambiente, decide si se une a la calma o al desorden y aprende a responder mejor si el jugador actúa correctamente.

Dibujo Informal:



Nivel 3 – “Control Aéreo”

Vista: tipo radar (cenital).

Objetivo: evitar colisiones entre los aviones.

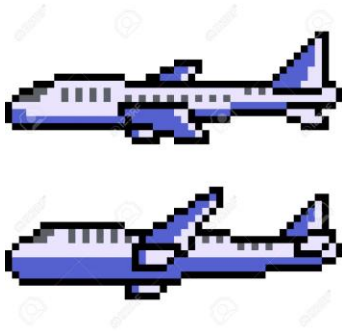
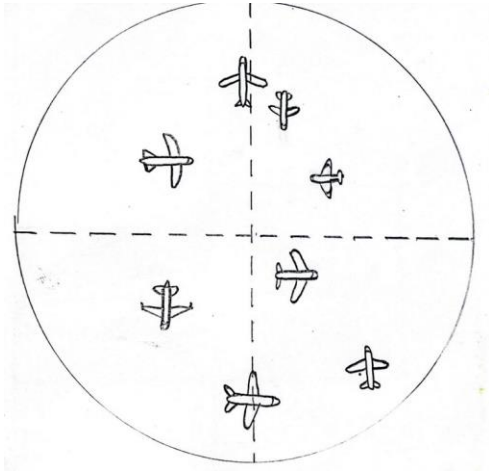
Interacciones: el jugador selecciona un avión y cambia su dirección.

Físicas:

1. Movimiento rectilíneo uniforme (aviones).
2. Movimiento circular (aviones en espera).
3. Movimiento acelerado (avión en descenso).

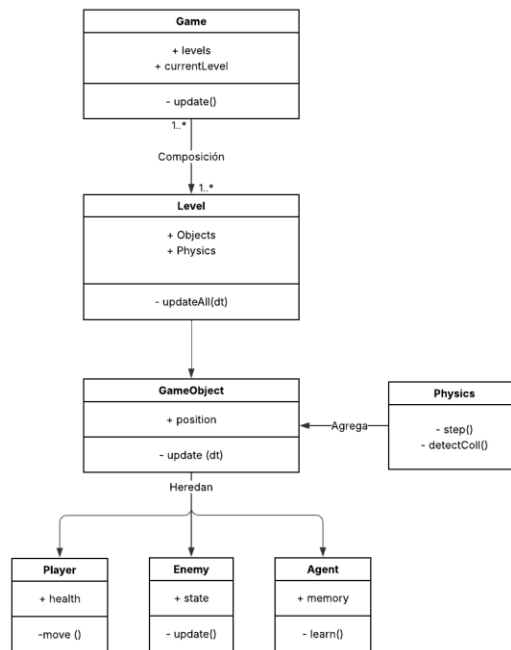
Agente autónomo: asistente que predice posibles choques y sugiere rutas seguras al jugador.

Dibujo Informal:



2. Diagrama de clases (nivel lógico)

Se presenta una versión simplificada de la capa lógica compatible con Qt. Incluye herencia, relaciones y uso de STL.



Relación	Tipo	Descripción
Game → Level	Composición	El juego tiene varios niveles.
Level → GameObject	Composición	Cada nivel contiene los objetos visibles y lógicos.
GameObject → Player, Enemy, Agent	Herencia	Todas las entidades comparten la misma interfaz base.
Level → Physics	Agregación	El nivel usa un módulo de física para actualizar posiciones.
Agent → GameObject	Dependencia	El agente percibe y reacciona a otros objetos del nivel.

En el diseño lógico del juego se emplean contenedores de la STL (Standard Template Library) para manejar de forma eficiente los objetos y niveles.

Cada nivel utiliza un `std::vector<std::shared_ptr<GameObject>>` para almacenar los elementos activos en pantalla, permitiendo acceso secuencial y gestión automática de memoria.

Además, se emplea `std::map<int, Level>` para organizar los niveles por identificador, y `std::unordered_map<int, GameObject*>` cuando se requiere acceso rápido por ID.

Estos contenedores simplifican la administración de memoria y hacen el código más seguro y mantenible dentro del entorno Qt.

3) Componentes del agente

Percepción: detecta algo (por ejemplo, fuego o un enemigo cerca).

Razonamiento: decide moverse o advertir.

Acción: ejecuta el movimiento o muestra un mensaje.

Aprendizaje: recuerda qué zonas fueron peligrosas o qué acciones funcionaron.

Diagrama de flujo:

